

Contents

spec-writing	1
When to use	1
The 12-section template (copy exactly)	1
The ID taxonomy (fixed alphabet)	3
Normative language discipline	3
Front-matter blockquote (copy literally)	4
§0 Intent rules (state explicitly)	4
Progressive-commit convention	4
Optional review-and-uplift workflow	4
Security check before publishing	5

spec-writing

Write a `SPEC.md` for a `labs/weekNN/chapterNN/` lab exactly as the known-good prior art (chapter1–chapter4) does, and no longer — every part below is observable, executable, and threaded to the `spec-review` pass.

When to use

- “create `SPEC.md` for ”
- “write the spec for the pipeline”
- “specify the `<interface>`”
- scaffold `labs/weekNN/chapterNN/SPEC.md`

Pair with `spec-review` when a review pass is needed; pair with `md2pdf-authoring` for the PDF rendering of the resulting `*.md`.

The 12-section template (copy exactly)

```
# SPECIFICATION - <System Name in Title Caps> (Domain Keywords, <entry-point-CLI>+, uv)

> - **Status:** v0.1 - draft for implementation review...
> - **Language:** Python 3.12 | <component stack> | CLI/GUI labels
> - **Curriculum source:** `curriculum/weekN/chapterN.md` ($X ... $Y) - a literal section list
> - **Scope of this document:** What the spec owns (the deterministic kernel, etc.)
> - **Normative language:** MUST/MUST NOT/SHALL/SHALL NOT = normative; SHOULD = strong; MAY
> - **Principle:** <one named invariant>, explained.

---

## 0. Intent and purpose
```

Why this subsystem exists; what the prior chapter(s) taught; where the deterministic vs probabilistic boundary sits. Include the central thesis quote when there is one.

1. Actors and goals

Actor	Goals

Name (<code>module.py</code>) Behavior contract (one sentence), with a parenthetical single-pr disclaimer where applicable	

2. Requirements (intent, high level)

ID	Statement
--	-----
R-01	Observable behavior

Requirements are **observable-behavior**-form, numbered R-01..R-NN. Map each to a curriculum

3. Behavior and state model

3.1 lifecycle scope (index-time / query-time, or eval-time, etc.); 3.2 the executable flow (ASCII box diagram `+`-`/`-`-``); 3.3 the durable artifacts pipeline.

4. Interfaces / contracts

C-01 ... ### C-NN

Each contract carries a fenced code block with a PINNED `@dataclass` or JSON-shape or module/gated-list; note the in-scope inversions (`(F-000)`). Never paste the whole contract t - only the behavioral pin.

5. Interface specification

5.1 CLI (`<cmd>`), primary surface

5.2 GUI, optional (`<cmd>-gui`)

CLI subcommand table columns: `Subcommand / Behavior / Exit` (usage errors exit `2`).

6. Invariants (must hold in every valid implementation)

ID	Invariant
--	-----
I-001	Zero-/negative-denominator fallbacks, byte-determinism, etc.

7. Constraints (precise and measurable)

ID	Constraint
--	-----

```
| **K-01** | All usage errors exit `2` (universal); others explicit |
```

8. Edge cases and failure semantics

```
| ID | Case | Semantics |
| -- | ---- | - - - - - |
| **E-01** | <input defect> | <deterministic outcome> |
```

9. Acceptance criteria, tests, and evals

9.N {group}

```
- **T-NN** description (ASCII text; KEEP `T-NN` unique - never collide ids)
```

10. Dependencies and environment

```
`Python 3.12 + uv`; libraries; optional GUI/dev group; host prerequisites (ollama pull ..., op
```

11. Traceability matrix (id → where realized)

ASCII text block - `ID --> module (behavior) --> test` - one row per R/C/I/E/K/T id from above

The ID taxonomy (fixed alphabet)

Prefix	Meaning	Example
R-nn	Requirement	R-14
C-nn	Contract (interface/data)	C-08
I-nn	Invariant	I-005
K-nn	Constraint	K-03
E-nn	Edge case / failure semantics	E-12
T-nn	Test / acceptance	T-09
O-n	Optionality (e.g. O-1 mock path, O-3 channel)	O-1
F-nnn	Spec-review finding	F-012 (only used after a review exists)

Papers MUST be uniquely numbered within their own family — collisions are the P1 finding's headline (T-08a/T-08a → rename T-08a/T-08c).

Normative language discipline

- MUST / MUST NOT / SHALL / SHALL NOT — the normative verbs.
- SHOULD — a strong recommendation (a violation triggers a documented warning, not a gate).

- MAY — an optional behavior (a gate MAY exist under some flag).
- Do not emit WILL, SHOULD BE ABLE TO, CAN in normative rows.

Front-matter blockquote (copy literally)

```
> - **Status:** ...
> - **Language:** ...
> - **Curriculum source:** ...
> - **Scope of this document:** ...
> - **Normative language:** ...
> - **Principle:** ...
```

The Curriculum source contains a literal curriculum section list like (§1 ... §38), not a bare filename.

§0 Intent rules (state explicitly)

- A one-sentence **thesis quote** of the surrounding chapter ("The evaluation suite is the bridge ..."), drawn word-for-word from curriculum/weekN/chapterN.md when available.
- Where the deterministic probabilistic boundary sits (which module owns which).
- A **Relationship to chN** paragraph when the lab extends a prior chapter.
- The **curriculum mapping** (rule → section list).

Progressive-commit convention

Write the SPEC in this order of commits, each commit message prefixed docs(chN): and, for the very first skeleton, chN: (as chapter4 did):

```
git add SPEC.md && git commit -m "chN: scaffold <lab-name> + instructions.txt; SPEC.md front
git add SPEC.md && git commit -m "docs(chN): SPEC.md §2 requirements R-01..R-NN"
git add SPEC.md && git commit -m "docs(chN): SPEC.md §3 state model + §4 contracts"
git add SPEC.md && git commit -m "docs(chN): SPEC.md §5 interfaces + §6 invariants + §7 cons
```

After any accidental glyph/ASCII cleanup, docs(chN): commit again. The review cycle: review(chN): per report-chapter made via the spec-review skill, and bump the version header with the findings reintegrated as fix(chN):.

Optional review-and-uplift workflow

If spec-review runs, follow the uplift campaign: apply P0 findings first, then P1, then commit docs(chN): bump SPEC.md v0.1->v0.2 (P0/P1 ...; P2 hygiene deferred). Or only apply them — deferring P2 findings is expressly acceptable.

Security check before publishing

- **Every ID unique** within its letter family (no T-08a collision across sections).
- **Every curriculum § referenced** actually exists in `curriculum/weekN/chapterN.md`.
- **Normative verbs only** (MUST/SHALL/SHOULD; no WILL/CAN in normative rows).
- **Front-matter blockquote present, exactly one** (6 named fields, exact list above).
- **12-S template order** preserved (Intent→Actors→Requirements→Behavior→Contracts→Interfaces→Invariants→Constraints→Edges→Tests→Deps→Traceability).
- **Traceability rows** mention literal module/behavior/test ids.